



Compression

Use compression to lower system requirements and save on infrastructure costs.

Overview

Weaviate stores vector representations, also called "vector embeddings" or simply "vectors." Each element of a vector embedding is a dimension. The values are commonly stored as 32 bit floats. A single vector with 1536 dimensions uses about 6 KB of storage.

When collections have millions of objects, the total storage size for the vectors can be very large. The vectors are not just stored, they are indexed as well. This index can also grow very large. The resource costs to host the vectors and the index can be expensive.

Weaviate creates indexes to search the vector space for your collection. The default vector index is a [Hierarchical Navigable Small World \(HNSW\)](#) index. This data structure includes the vectors as well as an index graph. HNSW indexes allow fast vector searches while maintaining excellent recall, but they can be expensive to use because they are stored in RAM memory.

Updating the default index configuration settings can result in significant cost savings and even performance improvements. In many cases, you can use compression or a different index type to change the way Weaviate stores and searches your data, and still maintain high levels of recall.

This page discusses compression algorithms. For more on indexes, see [Vector indexing](#).

Compression algorithms

These compression algorithms are available:

- [Product Quantization \(PQ\)](#) PQ reduces the size of the vector embedding in two ways. PQ trains on your data to create custom segments. PQ creates segments to reduce the number of dimensions, and segments are stored as 8 bit integers instead of 32 bit floats. Compared to dimensions, there are fewer segments and each segment is much smaller than a single dimension.

The PQ compression algorithm is [configurable](#). You control the number of segments, segment granularity, and the size of the training set.

- [Scalar Quantization \(SQ\)](#) SQ reduces the size of each vector dimension from 32 bits to 8 bits. SQ trains on your data to create custom buckets for each dimension. This training helps SQ to preserve data characteristics when it maps information from the 32 bit dimensions into 8 bit buckets.

- **Binary Quantization (BQ)** BQ reduces the size of each vector dimension to a single bit. This compression algorithm works best for vectors with high dimensionality.

When you compress vectors, the quality of your search results depends heavily on the characteristics of the uncompressed vectors. The embedding model produces these vectors. Therefore the embedding model is key to retrieval performance of the compressed vectors. Before moving to production, experiment with different compression settings and embedding models and review the model documentation to find the combination that works best with your data set.

Compression considerations

Before choosing a compression algorithm, consider the **underlying vector index type**. The index type determines which compression algorithms you can use. Some compression algorithms aren't available with some index types.

The vectorizer that you use to create your object vectors may also limit your compression choices. For example, some embedding models are tuned specifically for BQ compression.

Performance and cost are also important considerations. See **Cost, recall, and speed** for details.

Underlying vector index

This table shows the compression algorithms that are available for each index type.

Compression type	HNSW index	Flat index	Dynamic index
PQ	Yes	No	Yes
SQ	Yes	No	Yes
BQ	Yes	Yes	Yes

The **dynamic index** is new in v1.25. This type of index is a **flat index** until a collection reaches a threshold size. When the collection grows larger than the threshold size, the default is 10,000 objects, the collection is automatically reindexed and converted to an HNSW index.

Cost, recall and speed

Performance includes speed and recall. In real world systems, these factors have to be balanced against cost. As you develop familiarity with your application, you can tune Weaviate to match your project and budget requirements.

Cost

These compression algorithms have different functional tradeoffs, but they all help to control costs the same way. They reduce the size of the vectors so the indexes are smaller. Smaller indexes need less resources so you save money.

Compressed indexes use less RAM when they are loaded into memory, however they also use more disk space than uncompressed vectors. Weaviate stores the uncompressed vector and the compressed vector index. This means increased disk storage costs. However, since the cost of RAM is orders of magnitude higher than the cost of disk, the overall cost to use a compressed index is much lower than the cost of using an uncompressed index.

The cost savings are most visible with in-memory indexes such as HNSW. More RAM reduction means less cost.

- PQ compressed vectors typically use 85% less memory than uncompressed vectors.
- SQ compressed vectors use 75% less memory than uncompressed vectors.
- BQ compressed vectors use 97% less memory than uncompressed vectors.

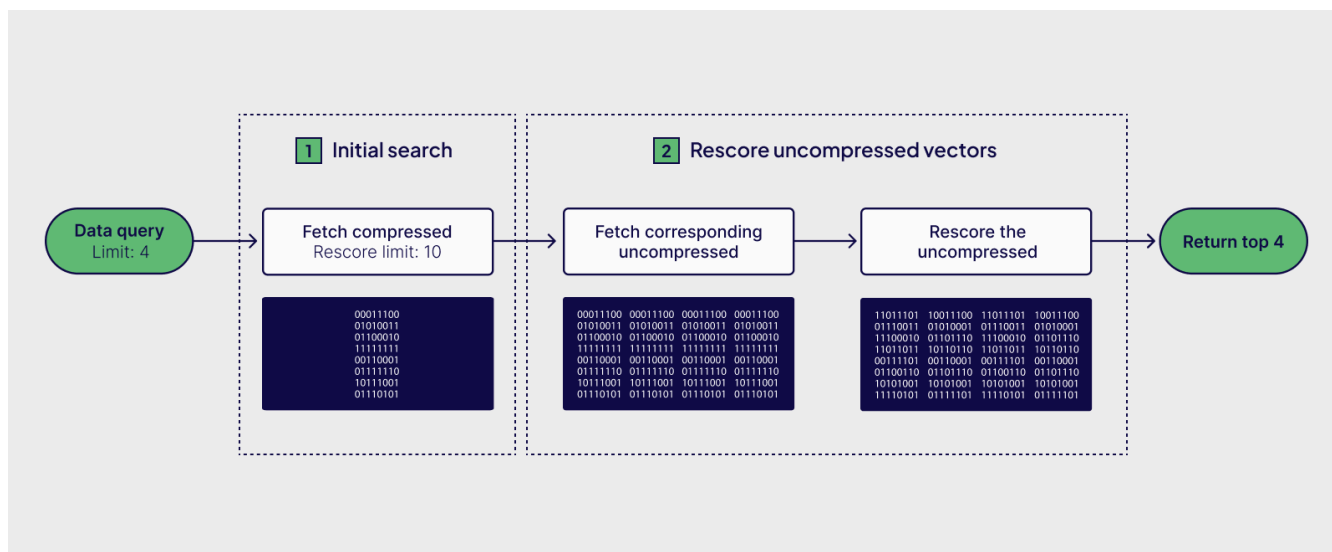
An HNSW index comprises a connection graph as well as the vectors. Quantization methods reduce the size of the vectors, but does not affect the size of the graph. As a result the overall reduction in memory usage is less than the reduction in vector size, but still significant.

Recall

Recall measures how well an algorithm finds true positive matches in a data set.

A compressed vector has less information than the corresponding uncompressed vector. An uncompressed vector that would normally match a search query might be missed if the target information is missing in the compressed vector. That missed match lowers recall.

To improve recall with compressed vectors, Weaviate over-fetches a list of candidate vectors during a search. For each item on the candidate list, Weaviate fetches the corresponding uncompressed vector. To determine the final ranking, Weaviate calculates the distances from the uncompressed vectors to the query vector.



The rescoring process is slower than an in-memory search, but since Weaviate only has to search a limited number of uncompressed vectors, the search is still very fast. Most importantly, rescoring with the uncompressed vectors greatly improves recall.

The search algorithm uses over-fetching and rescoring so that you get the benefits of compression without losing the precision of an uncompressed vector search.

Query speed

Each compression algorithm has its own characteristics with regard to speed.

- PQ indexes have response rates that approach the response rates of uncompressed indexes when recall reaches 97 percent and higher. At those levels of recall, the **speed profile** for PQ compressed indexes matches the profile for uncompressed indexes.
- BQ uses fast, bitwise calculations. BQ's bitwise calculations are extremely efficient. Efficient calculations are important because the flat index relies on brute-force search so it is calculation intensive. Searches of BQ compressed vectors can be as much as **10 to 20 times as fast** as similar searches of uncompressed vectors and with equivalent rates of recall. BQ is sensitive to the underlying data. If you use a flat index, evaluate BQ compression to verify the performance improvements with your data set.
- SQ significantly improves search speeds. SQ is new in v1.26. It is faster than PQ, perhaps 3 to 4 times as fast as searching uncompressed vectors. SQ has a higher dimensional resolution than BQ that helps recall. Look for an upcoming blog post that discusses the tradeoffs with SQ compression.

SQ and BQ both have optional vector caches. Use these configurable caches to load frequently used, uncompressed vectors into memory to improve overall search times.

Import speed

Importing and compressing vectors takes slightly longer than importing uncompressed vectors, but this is a one time cost. In contrast, loading a compressed index into memory is faster since there is less data

to load. This means restarts are faster.

Starting in v1.22, Weaviate has an optional, [asynchronous indexing](#) feature which effectively speeds up the import process. Consider enabling asynchronous indexing to improve imports.

Activate compression

- BQ and SQ have to be enabled when you create the collection.
- PQ and SQ both require training data before they begin to compress data.
 - PQ's training step defines centroids for each segment.
 - SQ's training step determines the minimum and maximum values for bucket boundaries.
 - PQ and SQ both begin to compress data after you have imported a large enough training set and the training step is complete.
- If you have async indexing and [AutoPQ enabled](#), PQ compression can be enabled anytime. If AutoPQ is not enabled, you should only enable PQ after you have imported enough objects to [train the algorithm](#).
- BQ doesn't require a training step.

Recommendations

Most applications benefit from compression. The cost savings are significant. In [Weaviate Cloud](#), for example, compressed collections can be more than 80% cheaper than uncompressed collections.

- If you have a small collection that uses a flat index, consider a BQ index. The BQ index is 32 times smaller and much faster than the uncompressed equivalent.
- Most users with medium to large data sets should consider SQ compression. SQ compressed vectors are one quarter the size of uncompressed vectors. Searches with SQ are faster than searches with uncompressed vectors. Recall is similar to uncompressed vectors.
- If you have a very large data set or specialized search needs, consider PQ compression. PQ compression is very configurable, but it requires more expertise to tune well than SQ or BQ.

For collections that are small, but that are expected to grow, consider a dynamic index. In addition to setting the dynamic index type, configure the collection to use BQ compression while the index is flat and SQ compression when the collection grows large enough to move from a flat index to an HNSW index.

Related pages

For more information, see these documentation pages and blog posts.

Documentation pages

To enable compression, follow the steps on these pages:

- [Product quantization \(PQ\)](#)
- [Scalar quantization \(SQ\)](#)
- [Binary quantization \(BQ\)](#)

For more documentation details, see:

- [Compression discussion](#)

Blog posts

For in-depth discussions, see:

- [PQ and memory reduction](#)
- [BQ and memory reduction](#)
- [PQ and HNSW explained](#)

Pricing calculator

To review Weaviate Cloud pricing for compressed and uncompressed vectors, see:

[Weaviate cloud pricing calculator](#)

Questions and feedback

If you have any questions or feedback, let us know in the [user forum](#).

- [Technical questions](#)
- [Documentation feedback](#)

 [Edit this page](#)